

1.2.2 Продвинутые типы prompts

Модуль 1. Основы prompt engineering

Оглавление

1. Введение	3
2. Метод self-consistency	5
3. Generated Knowledge Prompting.....	6
4. Prompt Chaining	9
5. Tree of Thoughts	10
6. Automatic Reasoning prompt	11
7. Retrieval Augmented Generation	12
8. Заключение	14

1. Введение

Уважаемые коллеги! Сегодня мы приступаем к изучению продвинутых техник промптинга, которые позволят вам значительно расширить возможности работы с большими языковыми моделями. В современном мире искусственного интеллекта умение эффективно взаимодействовать с моделями становится ключевым навыком для специалистов различных областей.

Мы рассмотрим шесть наиболее перспективных методов: self-consistency, Generated Knowledge Prompting, prompt chaining, tree of thoughts, automatic reasoning и Retrieval Augmented Generation. Каждый из этих методов решает специфические задачи и помогает преодолеть ограничения базовых подходов к промптингу.

Self-consistency позволяет повысить достоверность генерируемых ответов, Generated Knowledge Prompting помогает модели создавать и использовать новые знания, а prompt chaining структурирует процесс генерации, делая его более предсказуемым и управляемым. Tree of thoughts и automatic reasoning расширяют способности моделей к логическому мышлению, а Retrieval Augmented Generation позволяет обогащать ответы актуальной информацией из внешних источников.

В ходе лекции мы подробно разберем каждый метод, рассмотрим примеры их практического применения и обсудим, в каких случаях наиболее эффективно использовать тот или иной подход. К концу занятия вы получите комплексное представление о современных техниках продвинутого промптинга и сможете применять их в своей профессиональной деятельности.

Приготовьтесь к глубокому погружению в мир продвинутых методов работы с языковыми моделями. Уверен, что полученные знания станут важным инструментом в вашем профессиональном арсенале.

2. Метод self-consistency

Рассмотрим метод **self-consistency**, разработанный Wang et al. в 2022 году. Этот метод позволяет значительно повысить точность ответов больших языковых моделей за счет их взаимной проверки.

Представьте ситуацию: вы спрашиваете у модели один и тот же вопрос несколько раз, и каждый раз получаете немного разные ответы. Как определить, какой из них правильный? Именно для решения этой проблемы и был создан метод self-consistency.

Давайте разберем процесс пошагово. Сначала вы формулируете вопрос и получаете первый ответ от модели. Затем вы задаете тот же вопрос еще несколько раз (обычно 3-5), каждый раз получая новый вариант ответа. Важно при этом немного менять формулировку вопроса или контекст, чтобы получить максимально разнообразные ответы.

После получения всех вариантов вы анализируете их на предмет согласованности. Обратите внимание на то, какие утверждения повторяются в разных ответах, а какие противоречат друг другу. Чем чаще встречается определенное утверждение, тем выше вероятность его достоверности.

На следующем этапе вы формируете итоговый ответ, основываясь на согласованных частях всех вариантов. При этом противоречия между ответами могут указывать на неоднозначность вопроса или недостаточную информированность модели.

Метод особенно эффективен при работе с задачами, где требуется высокая точность ответов: медицинские диагнозы, финансовые расчеты, научные исследования. Однако важно помнить, что даже при использовании этого метода нельзя гарантировать абсолютную точность – мы лишь повышаем вероятность получения правильного ответа.

При применении метода рекомендую следовать нескольким правилам:
Во-первых, формулируйте вопросы максимально четко и конкретно.

Во-вторых, внимательно анализируйте не только конечные ответы, но и логику их получения.

В-третьих, учитывайте контекст задачи и специфику предметной области.

И наконец, не ограничивайтесь только согласованными утверждениями – иногда противоречия могут указывать на важные нюансы, требующие дополнительного изучения.

Практическое применение метода требует некоторого времени и внимания к деталям, но результаты того стоят – вы получаете более надежные и обоснованные ответы от модели.

Надеюсь, теперь вам стало понятнее, как использовать метод self-consistency для повышения качества работы с языковыми моделями. При правильном применении этот метод может стать мощным инструментом в вашей исследовательской и профессиональной деятельности.

3. Generated Knowledge Prompting

Теперь рассмотрим метод **Generated Knowledge Prompting (GKP)**, предложенный Liu et al. в 2022 году. Этот метод представляет собой инновационный подход к обучению больших языковых моделей через создание дополнительных обучающих примеров.

Основная идея GKP заключается в том, что вместо простого использования существующих данных для обучения модели, мы генерируем новые обучающие примеры на основе уже имеющихся знаний модели. Это позволяет значительно расширить обучающий набор данных и улучшить качество обучения.

Процесс использования GKP состоит из нескольких этапов. Сначала необходимо обучить базовую модель на имеющемся наборе данных. Затем, используя эту модель, генерируются новые примеры, которые дополняются

к исходному набору данных. После этого модель переобучается уже на расширенном наборе данных.

Важно отметить, что генерируемые примеры должны быть релевантны исходной задаче и не содержать противоречивой информации. Для этого рекомендуется использовать несколько итераций генерации и тщательно проверять качество сгенерированных данных.

Практическое применение GKP особенно эффективно в случаях, когда у вас ограниченный набор обучающих данных или, когда вы хотите улучшить обобщающую способность модели. Этот метод позволяет существенно повысить качество обучения без значительных затрат на сбор дополнительных данных.

Рассмотрим практический пример использования метода Generated Knowledge Prompting на задаче классификации текстов. Предположим, у нас есть простая задача определения тональности отзывов о ресторанах. У нас есть небольшой набор данных из трех примеров:

1. "Еда была вкусной, обслуживание отличное" - положительный
2. "Повар испортил блюдо, официанты грубые" - отрицательный
3. "Средний ресторан, ничего особенного" - нейтральный

Используя метод GKP, мы можем сгенерировать дополнительные обучающие примеры следующим образом:

1. Берём базовую модель и подаём ей первый пример: "Еда была вкусной, обслуживание отличное"

Модель генерирует новый пример: "Порции большие, атмосфера приятная" - положительный

2. Для второго примера: "Повар испортил блюдо, официанты грубые"

Модель создаёт: "Задержали заказ на час, десерты несвежие" - отрицательный

3. Для третьего примера: "Средний ресторан, ничего особенного"

Модель предлагает: "Цены высокие, кухня обычная" - нейтральный

Теперь наш обучающий набор расширился до шести примеров, что значительно улучшит качество обучения модели. Важно отметить, что сгенерированные примеры должны быть разнообразными и не просто копией исходных данных.

При генерации новых примеров можно использовать различные стратегии:

- Менять порядок слов
- Заменять синонимы
- Добавлять новые детали
- Изменять структуру предложений

Такой подход особенно полезен, когда у вас ограниченный набор данных или, когда вы хотите улучшить обобщающую способность модели. При этом важно проверять качество сгенерированных примеров и их релевантность исходной задаче.

В результате применения GKP модель получает больше разнообразных примеров для обучения, что помогает ей лучше понимать контекст и улучшать точность классификации.

При использовании GKP следует учитывать несколько важных моментов. Во-первых, необходимо тщательно настраивать параметры генерации, чтобы избежать создания избыточно похожих примеров. Во-вторых, рекомендуется периодически проверять качество сгенерированных данных, так как модель может начать генерировать нерелевантные или ошибочные примеры.

В заключение хочу отметить, что метод Generated Knowledge Prompting представляет собой мощный инструмент для улучшения качества обучения языковых моделей. При правильном применении он может

значительно повысить эффективность обучения и качество получаемых результатов.

4. Prompt Chaining

Следующий важный метод работы с языковыми моделями – это **Prompt Chaining**.

Prompt Chaining – это техника последовательной обработки информации, при которой один промпт логически продолжает предыдущий, формируя цепочку взаимодействий с моделью. Представьте, что вы задаете модели ряд связанных вопросов или ставите последовательность задач, где каждый следующий шаг зависит от результата предыдущего.

Принцип работы прост: вы начинаете с базового промпта, получаете ответ, затем используете его как основу для следующего запроса. Это позволяет модели лучше понимать контекст и генерировать более точные и релевантные результаты.

Давайте рассмотрим, в каких случаях стоит применять Prompt Chaining:

1. При решении сложных многоэтапных задач, где важен последовательный подход
2. Когда нужно получить детальный анализ с постепенным углублением в тему
3. В случаях, требующих поэтапной обработки больших объемов данных
4. При работе с проектами, где важна точность и последовательность шагов
5. Когда необходимо избежать потери контекста при длительной коммуникации

Важно помнить, что при использовании Prompt Chaining нужно тщательно следить за передачей информации между запросами и правильно формулировать каждый следующий шаг на основе предыдущего результата. Это требует определенной практики и понимания логики работы модели.

Рекомендую начинать с простых цепочек из 2-3 промптов, постепенно усложняя их по мере освоения техники. Помните, что правильно построенная цепочка промптов может значительно повысить качество получаемых результатов и сделать работу с моделью более эффективной.

При этом стоит учитывать ограничения: слишком длинные цепочки могут привести к накоплению ошибок, а также увеличить время обработки запросов. Поэтому важно находить оптимальный баланс между количеством шагов в цепочке и достигаемым результатом.

5. Tree of Thoughts

Теперь давайте рассмотрим инновационный метод под названием **Tree of Thoughts**, разработанный Яо в 2023 году.

Tree of Thoughts представляет собой алгоритм, который позволяет искусственному интеллекту более эффективно решать сложные задачи путем построения дерева рассуждений. Принцип работы заключается в следующем: система генерирует множество возможных решений (мысли), затем оценивает их и выбирает наиболее перспективные для дальнейшего развития.

Процесс можно представить, как дерево, где каждый узел – это отдельная мысль или решение. От начальной точки (корня дерева) алгоритм разветвляется, создавая новые варианты решений. При этом система постоянно анализирует промежуточные результаты и отбрасывает менее эффективные пути.

Алгоритм особенно эффективен в следующих случаях:

- При решении задач с множеством возможных решений
- В ситуациях, где требуется творческий подход
- При работе с открытыми вопросами, не имеющими однозначного ответа
- В задачах, требующих пошагового анализа и принятия решений

Главное преимущество метода заключается в том, что он позволяет модели не просто находить решение, а осознанно выбирать наилучший путь к нему, учитывая контекст и промежуточные результаты. Это делает его особенно полезным для задач, требующих глубокого понимания и логического мышления.

Важно отметить, что эффективность алгоритма напрямую зависит от качества оценки промежуточных решений и способности модели правильно определять наиболее перспективные направления развития мысли.

В практических задачах этот метод можно применять при написании текстов, решении математических задач, разработке стратегий, анализе данных и в других областях, где требуется системный подход к поиску решения.

6. Automatic Reasoning prompt

Следующий важный инструмент в работе с большими языковыми моделями - **Automatic Reasoning prompt**. Это специальный формат запроса, который помогает модели более эффективно решать сложные задачи, требующие логического мышления и пошагового анализа.

Принцип работы Automatic Reasoning prompt заключается в том, что вы структурируете запрос таким образом, чтобы модель могла последовательно анализировать информацию, делать логические выводы и приходить к обоснованному решению. В отличие от обычных запросов, здесь вы задаете четкую схему рассуждения, которую модель должна следовать.

Структура такого запроса обычно включает:

- Четкое описание задачи
- Перечисление необходимых шагов для решения
- Формулу или шаблон для логического вывода
- Требование к формату ответа

Например, если нужно решить математическую задачу, вы можете попросить модель сначала определить тип задачи, затем перечислить необходимые формулы, пошагово применить их и только потом дать ответ.

Главное преимущество этого подхода в том, что он помогает избежать ошибок в рассуждениях и обеспечивает более прозрачное решение задачи. Модель не просто дает ответ, а показывает весь процесс его получения, что особенно важно при обучении или проверке решений.

Помните, что правильно составленный Automatic Reasoning prompt значительно повышает качество получаемых результатов и помогает избежать нелогичных или ошибочных выводов. Этот подход особенно полезен при работе с сложными задачами, где важен не только конечный ответ, но и процесс его получения.

7. Retrieval Augmented Generation

И напоследок давайте поговорим о важной концепции в работе с большими языковыми моделями - **Retrieval Augmented Generation (RAG)**.

Представьте, что вы готовитесь к экзамену. Можно просто заучить всю информацию из учебников (это как работа базовой LLM), а можно еще и воспользоваться свежими материалами из интернета, научными статьями и дополнительными источниками (это как RAG).

RAG - это метод, который позволяет языковой модели не только использовать свои встроенные знания, но и дополнять их актуальной информацией из внешних источников данных. Проще говоря, это, как если

бы у модели был доступ к огромной библиотеке, из которой она может брать актуальную информацию для ответа.

Принцип работы RAG довольно прост: когда поступает запрос, система сначала ищет релевантные документы в базе данных, затем извлекает из них нужную информацию и только потом генерирует ответ, опираясь как на свои знания, так и на извлеченные данные.

Когда же стоит использовать RAG? Вот несколько ключевых ситуаций:

- Когда требуется работа с актуальными данными, которые быстро меняются (новости, статистика, цены)
- При необходимости доступа к специфическим знаниям из определенных источников
- Когда нужно обеспечить точность и достоверность информации
- В случаях, где важна актуальность данных на момент запроса
- При работе с профессиональными базами данных (медицина, юриспруденция)

Важно понимать, что RAG не заменяет базовую LLM, а дополняет её. Это как опытный специалист, который не только полагается на свои знания, но и сверяется с профессиональной литературой перед принятием важного решения.

Однако у метода есть и ограничения: требуется качественная система поиска информации, необходимо обеспечить безопасность данных и защиту от утечки конфиденциальной информации. Также нужно учитывать время на поиск и обработку данных, что может замедлить генерацию ответа.

В современных условиях RAG становится все более важным инструментом, особенно в таких областях как:

- Корпоративные информационные системы
- Медицинские диагностические системы
- Юридические консультации

- Финансовые аналитики
- Образовательные платформы

Помните, что правильно настроенная система RAG может значительно повысить качество и актуальность генерируемых ответов, делая работу с LLM более эффективной и надежной.

8. Заключение

Дорогие коллеги, сегодня мы рассмотрели восемь ключевых методов продвинутого prompting'a, каждый из которых открывает новые возможности в работе с большими языковыми моделями.

Self-consistency позволяет повысить достоверность генерируемого контента, **Generated Knowledge Prompting** помогает создавать более информативные ответы, а **Function calling** расширяет возможности моделей за счет интеграции с внешними сервисами. **Структурированные промпты** и **prompt chaining** помогают организовать сложные задачи в понятную последовательность действий. **Tree of thoughts** позволяет модели мыслить более креативно и находить нестандартные решения, а автоматическое рассуждение помогает в решении сложных логических задач. И наконец, **Retrieval Augmented Generation** дает возможность обогащать ответы актуальной информацией из внешних источников.

Важно понимать, что эти методы не существуют изолированно друг от друга. Часто для достижения наилучшего результата требуется комбинировать различные подходы, адаптируя их под конкретную задачу. Помните, что мастерство в prompting'e приходит с практикой – чем больше вы экспериментируете с разными техниками, тем лучше понимаете их потенциал и ограничения.

Рекомендую вам начать применять изученные методы в своей работе, начиная с тех, которые кажутся наиболее подходящими для ваших задач.

Постепенно расширяйте свой инструментарий, изучая новые подходы и адаптируя их под свои потребности. Помните, что продвинутый prompting – это не просто набор техник, а искусство эффективного взаимодействия с языковыми моделями.

Благодарю за внимание. Надеюсь, полученные знания помогут вам в дальнейшей работе с большими языковыми моделями.